Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives, contrôle de flot, principe DRY et prévention des boucles infinies

Synthèse exécutive : Ce cours pose les bases fondamentales des structures itératives (boucles) en algorithmique et en langage C. L'enjeu central est d'automatiser l'exécution répétée d'un bloc d'instructions sans duplication de code, conformément au principe DRY (Don't Repeat Yourself). Le cours détaille le fonctionnement, la syntaxe C stricte et les critères de choix entre les trois formes de boucles : 'while' (à pré-condition, 0 à N itérations), 'do-while' (à post-condition, 1 à N itérations garanties, essentielle pour la validation de saisie) et 'for' (à en-tête condensé pour itérations déterminées). Une attention rigoureuse est portée sur l'initialisation et la mise à jour des variables de contrôle pour éviter les boucles infinies, ainsi que sur les conventions de codage exigées à l'IUT d'Anglet (style Allman, tripartition et cartouche).

1. Utilité des structures itératives et principe DRY

En programmation algorithmique, la répétition d'instructions ne doit jamais résulter d'un copier-coller de lignes de code. Si l'on doit traiter 10 000 éléments d'un tableau ou réitérer une demande de saisie jusqu'à ce qu me la valeur soit valide, dupliquer le code détruit la maintenabilité du programme, alourdit inutilement le binaire exécutable et constitue une source massive d'erreurs. Les structures itératives (ou boucles) permettent d'exécuter de façon répétée un même bloc d'instructions tant qu'une condition logique (expression booléenne) reste vérifiée. Le point névralgique de la conception d'une boucle repose sur la gestion de sa fin. Une boucle robuste exige trois éléments indissociables : une initialisation correcte des variables de contrôle avant la boucle, une condition de maintien/sortie claire, et une mise à jour explicite de la variable de contrôle dans le corps de la boucle à chaque itération. Si le corps de la boucle n'altère pas les variables testées dans la condition, celle-ci conserve sa valeur initiale Vraie : le programme entre alors dans une 'boucle infinie', saturant le processeur et gelant l'application.

- Principe DRY (Don't Repeat Yourself) : Interdiction absolue de dupliquer manuellement des lignes de code pour répéter une tâche.
- Condition d'arrêt : Expression booléenne évaluée pour déterminer si la boucle doit continuer ou s'arrêter.
- Variable de contrôle / Compteur : Variable modifiée au sein de la boucle qui fait évoluer la condition vers l'arrêt.
- Boucle infinie : Dysfonctionnement majeur surviennent lorsque la condition d'arrêt reste indéfiniment Vraie par manque de mise à jour.

Cas concret enseignant : *Métaphore du tapis roulant d'une usine d'emballage : tant qu'un colis est détecté par le capteur optique (condition vraie), le bras robotisé attrape le colis et l'emballe (instruction). Dès que le capteur ne détecte plus de colis ou que le quota fixé est atteint (condition fausse), le robot s'arrête. Sans capteur fonctionnel ou sans mise à jour du compteur de colis, le robot continuerait à tourner à vide indéfiniment jusqu'à provoquer une surchauffe et détruire la machine.*

2. La boucle TANT QUE (while) : Pré-conditionnement

La structure 'while' est le choix algorithmique naturel lorsque le nombre exact d'itérations ne peut pas être déterminé à l'avance et dépend d'un événement dynamique externe (une saisie utilisateur, la lecture d'un fichier jusqu'à sa fin, ou l'attente d'un signal réseau). En langage C, sa syntaxe s'écrit : while (condition) { /* instructions */ } Puisque le contrôle s'effectue en amont du bloc, il est possible que les instructions internes ne s'exécutent jamais (0 itération) si la condition est fausse initialement. L'étudiant doit veiller à deux pièges fréquents : d'une part, l'omission de l'initialisation des variables de condition avant le while (conduisant à tester des valeurs indéterminées en mémoire) ; d'autre part, l'oubli de la modification de ces variables à l'intérieur de la boucle (provoquant une boucle infinie).

- Pré-conditionnement : L'expression booléenne est évaluée AVANT d'exécuter le bloc d'instructions.
- Plage d'itérations : De 0 à N passages. Si la condition est fausse dès le premier contrôle, le bloc n'est jamais exécuté.

• Initialisation préalable obligatoire : Toute variable intervenant dans la condition d'évaluation du while doit posséder une valeur connue avant le test.

Cas concret enseignant : *Exemple du contrôle de code PIN à un distributeur automatique de billets (DAB) : le système demande le code secret TANT QUE le code saisi est incorrect ET QUE le compteur d'essais est strictement inférieur à 3. Si l'usager saisit le bon code du premier coup, la condition devient fausse immédiatement et le corps de la boucle n'est pas réexécuté.*

3. La boucle FAIRE... TANT QUE (do-while) : Post-conditionnement

À la différence de la boucle 'while', la structure 'do-while' garantit que le bloc d'instructions sera exécuté au moins une première fois avant que la condition logique ne soit testée. En langage C, sa syntaxe s'écrit : do { /* instructions */ } while (condition); Cette boucle est spécifiquement adaptée à la saisie sécurisée de données (validation de bornes numériques) et à la gestion de menus interactifs. En effet, il est impossible d'évaluer la validité d'une donnée saisie par un utilisateur avant de lui avoir au moins demandé de la saisir une première fois. Le piège de syntaxe majeur réside dans l'oubli du point-virgule final juste après `while (condition);`, erreur classique qui bloque immédiatement la compilation.

• Post-conditionnement : L'expression booléenne est évaluée APRÈS chaque exécution du bloc d'instructions.

• Garantie d'exécution minimale : La boucle s'exécute obligatoirement au moins 1 fois (1 à N itérations).

• Point-virgule final impératif : La syntaxe C exige obligatoirement un ';' après la parenthèse fermante du while.

Cas concret enseignant : *Contrôle de saisie d'une note d'examen (entre 0 et 20) : 'On ne peut pas vérifier si la note entrée par un utilisateur est valide avant qu'il n'ait tapé la valeur au clavier. Il faut exécuter la demande de saisie au moins une fois, ce qui fait du do-while la structure idéale pour le contrôle de saisie.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en réunissant sur une ligne d'en-tête unique l'ensemble de la logique d'itération. En langage C, la syntaxe s'écrit : for (initialisation ; condition ; incrémentation) { /* instructions */ } Le fonctionnement interne respecte un ordre strict : 1) Exécution unique de l'initialisation ; 2) Évaluation de la condition booléenne ; 3) Si la condition est Vraie, exécution du corps de la boucle ; 4) Exécution de l'incrémentation/décrémentation ; 5) Retour à l'étape 2. Bien que le C autorise l'omission de certaines parties dans l'en-tête (ex: `for(;;)` pour créer une boucle infinie), la norme d'écriture à l'IUT impose de renseigner explicitement les trois expressions pour garantir une lisibilité optimale.

• En-tête condensé : Centralise l'initialisation, la condition de maintien et l'incrémentation sur une seule ligne.

• Nombre d'itérations déterminé : Structure recommandée lorsque le nombre exact de répétitions est connu avant d'entrer dans la boucle.

• Portée (scope) : Possibilité de limiter la durée de vie de la variable d'itération en la déclarant dans l'en-tête (standard C99).

Cas concret enseignant : *Métaphore des 5 tours de piste de l'athlète sur un stade : 'Avant d'attaquer la course, vous savez que vous devez accomplir exactement 5 tours. Vous préparez votre compteur au tour 1 (initialisation), vous contrôlez que vous n'avez pas dépassé le 5e tour (condition), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (incrémentation).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Si la variable de contrôle 'i' n'est pas modifiée au sein du corps du while, l'expression booléenne testée dans la condition conserve exactement la même valeur qu'à l'entrée. Si elle était vraie au départ, elle reste vraie indéfiniment. Le programme s'engouffre alors dans une boucle infinie, monopolisant 100% des ressources processeur allouées sans jamais parvenir à l'instruction de sortie.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, sur le plan strictement algorithmique et logique, toute boucle 'for' peut être réécrite sous la forme d'une boucle 'while' en plaçant l'initialisation juste avant la boucle et l'incrémentation tout à la fin du bloc d'instructions. Cependant, lorsque le nombre d'itérations est fixe et connu au départ, la boucle 'for' est une exigence de lisibilité et de propreté logicielle.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers strictly positifs (1 + 2 + ... + N). Le programme doit impérativement sécuriser la saisie de N au moyen d'une boucle 'do-while' pour refuser toute valeur inférieure ou égale à 0, puis calculer la somme cumulative à l'aide d'une boucle 'for'. Le programme doit être rédigé selon la norme stricte de l'IUT d'Anglet (cartouche d'en-tête, tripartition CONSTANTES / VARIABLES / TRAITEMENTS, flèches de flot de données, style Allman).

Démarche de résolution :
Étape 1 : Déclaration des variables dans la tripartition '// VARIABLES' (n pour la limite, somme pour le cumul, i pour l'itération). Étape 2 : Construction de la boucle 'do-while' avec condition de maintien (n <= 0) pour forcer l'utilisateur à saisir un entier strictement positif. Étape 3 : Initialisation explicite du cumul somme à 0 avant le début du calcul. Étape 4 : Mise en place de la boucle 'for' débutant à i = 1, s'arrêtant lorsque i > n, et incrémentant i de 1 à chaque tour. Étape 5 : Affichage formaté du résultat via printf et retour 0 de fin de programme.

```c
/*
 Nom du fichier : somme_n_entiers.c
 Auteur : Matéi de Saint Leger
 Date : 12/11/2026
 Objectif : Somme des N premiers entiers avec validation do-while et calcul for
 Matière : R1.01 - IUT de Bayonne et du Pays Basque (Campus Anglet)
*/

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    // CONSTANTES

    // VARIABLES
    int n;     // Nombre d'entiers à sommer (fourni par l'utilisateur)
    int somme; // Accumulateur de la somme
    int i;     // Compteur de la boucle for

    // TRAITEMENTS
    // 1. Saisie sécurisée de N avec validation do-while
    do
    {
        printf("Entrez un entier N (> 0) : ");
        // (clavier) >> saisir >> n
        scanf("%d", &n);

        if (n <= 0)
        {
            printf("Erreur : N doit etre strictement positif !\n");
        }
    }
    while (n <= 0);

    // 2. Calcul de la somme cumulative avec la boucle for
    somme = 0;
    for (i = 1; i <= n; i++)
    {
        somme = somme + i;
    }

    // 3. Affichage du résultat final
    // (somme, n) >> afficher >> (ecran)
    printf("La somme des %d premiers entiers est : %d\n", n, somme);

    return 0;
}
```

Résultat : Pour une saisie avec N = 5 : - La boucle 'do-while' valide N (car 5 > 0 est vrai, donc (5 <= 0) est faux, fin du do-while). - La boucle 'for' exécute 5 itérations : * i = 1 : somme = 0 + 1 = 1 * i = 2 : somme = 1 + 2 = 3 * i = 3 : somme = 3 + 3 = 6 * i = 4 : somme = 6 + 4 = 10 * i = 5 : somme = 10 + 5 = 15 - Le programme affiche : 'La somme des 5 premiers entiers est : 15'. Si l'utilisateur avait saisi -3 au départ, le do-while aurait affiché un message d'erreur et réclamé la saisie.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' évalue sa condition de maintien AVANT l'exécution du bloc (pré-condition, de 0 à N itérations), alors que la boucle 'do-while' évalue sa condition APRÈS l'exécution du bloc (post-condition), garantissant au moins 1 exécution.

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès la première évaluation ?

R : Le corps de la boucle 'for' n'est jamais exécuté (0 itération). Le programme saute directement à l'instruction qui suit la fermeture de la boucle.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule final ';' immédiatement après la parenthèse fermante de l'instruction `while (condition);`.